RobotArm

軟件設計文件

版本: v0.1

日期: 2026-09-16

作者: Ben Yung (必思系統)

狀態: 草稿待審閱

品牌中立嘅多機械人控制框架

---

title: "RobotArm 軟件設計文件"

version: "v0.1"

date: "2026-09-16"

author: "Ben Yung (必思系統)"

RobotArm 軟件設計文件

版本: v0.1

日期: 2026-09-16

作者: Ben Yung (必思系統)

狀態: 草稿待審閱

目錄

1. 執行摘要
2. 系統架構
3. 模組規格
4. API 參考
5. 使用案例
6. 專案結構
7. 開發路線圖
8. 風險評估
9. 成功準則

---

1. 執行摘要

1.1 專案概覽

RobotArm 係一個品牌中立的軟件框架,用作控制工業協作機械臂(collaborative robot)。由香港一間製造公司開發,主要支援多機械臂協調嘅工業自動化,以焊接應用為主要目標。

1.2 目標與宗旨

目標	說明
虛擬優先開發	可喺寫字樓完全開發同測試,唔需要實體機械人
品牌中立	透過統一 API 支援多個機械人品牌
多機械人協調	同時控制 2-4 個機械臂,支援同步動作
輕鬆部署	由虛擬切換去真實硬件只需改 1-2 行 code
應用靈活	支援焊接、上下料、組裝、檢測等

1.3 目標用戶

- **軟件開發者** — 喺寫字樓寫機械人控制程式
- **系統整合商** — 將方案部署到工廠
- **技術經理** — 規劃自動化項目

1.4 主要功能

- 虛擬機械人模擬,貼近真實機械人規格
- 多機械人管理,集中安全控制
- 模組化應用庫(焊接、上下料、組裝、檢測)
- 真實 SDK 適配器(ROKAE xCore、JAKA SDK)
- 最少依賴(numpy + Python 標準庫)

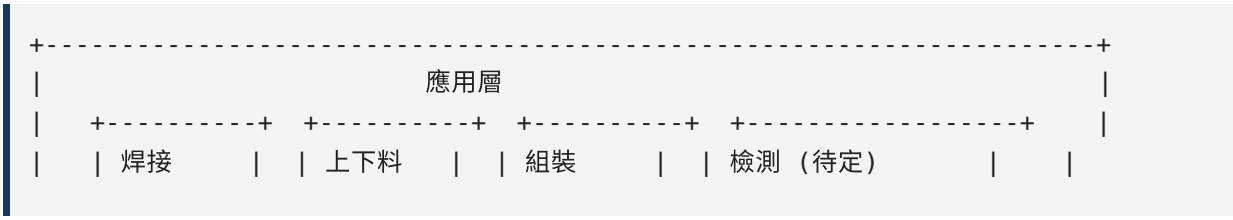
1.5 現有硬件

數量	品牌	負載	工作半徑	防護等級	角色
2	ROKAE CR7	7 kg	988 mm	IP67	主要焊接
1	JAKA Zu7	7 kg	819 mm	IP54	後備 / 輕量任務

---

2. 系統架構

2.1 整體架構



```
|   +----------+  +----------+  +----------+  +------------------+    |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                           核心層                                   |
|   +------------------+    +------------------+                     |
|   |  RobotManager    |<->|  VirtualRobot    |                     |
|   |  (多機械人管理)   |    |  (單一機械人)     |                    |
|   +------------------+    +------------------+                     |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                         品牌適配層                                  |
|   +------------------+  +------------------+  +----------------+    |
|   | ROKAE_CR7_適配器 |  | JAKA_Zu7_適配器   |  | 未來品牌       |    |
|   | (主要)           |  | (次要)            |  | (UR, ABB, ...) |    |
|   +------------------+  +------------------+  +----------------+    |
|         |                       |                                   |
|         v                       v                                   |
|   +------------------+  +------------------+                        |
|   | ROKAE xCore SDK  |  |  JAKA Python SDK |                        |
|   +------------------+  +------------------+                        |
+-------------------------------------------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                         工具層                                     |
|   +--------------+  +--------------+  +--------------------------+  |
|   | 運動學       |  |   安全       |  |  日誌                    |  |
|   +--------------+  +--------------+  +--------------------------+  |
+-------------------------------------------------------------------+
```

2.2 模組劃分

層	模組	職責
應用	`welding.py`	焊接路徑規劃、焊槍控制
應用	`pick_place.py`	夾爪控制、點對點動作
應用	`assembly.py`	精密動作、力控接口(預留)
應用	`inspection.py`	巡檢路徑、傳感器整合
核心	`virtual_robot.py`	模擬機械人,貼近真實 SDK API
核心	`robot_manager.py`	多機械人協調、集中緊急停止

層	模組	職責
品牌	`rokae_adapter.py`	ROKAE CR7 虛擬 + 真實封裝
品牌	`jaka_adapter.py`	JAKA Zu7 虛擬 + 真實封裝
工具	`kinematics.py`	正向運動學、位姿轉換
工具	`safety.py`	關節極限、工作範圍、防撞檢查
工具	`logger.py`	集中日誌配置

2.3 資料流向

```
用戶程式
    |
    v
應用模組 (例如焊接)
    |
    v
RobotManager / VirtualRobot (核心 API)
    |
    v
品牌適配器 (ROKAE / JAKA)
    |
    +-- 虛擬路徑: 純 Python 模擬 (numpy)
    |
    +-- 真實路徑: 廠商 SDK -> 網絡 -> 實體機械人
```

框架嘅設計令應用模組**唔需要知道**自己對住嘅係虛擬定真實機械人。唯一嘅分別係實例化邊個 class。

2.4 類別圖

```
                +----------------------+
                |   RobotManager       |
                +----------------------+
                | - robots: dict       |
                +----------------------+
                | + add_robot()        |
                | + connect_all()      |
                | + emergency_stop_all |
                | + sync_move()        |
                +----------+-----------+
                           | 擁有 0..N
                           v
                +----------------------+
                |   VirtualRobot       |
```

```
                +----------------------+
                | - robot_id: str      |
                | - brand: str         |
                | - model: str         |
                | - joints: array      |
                | - pose: array        |
                | - io_states: dict    |
                | - connected: bool    |
                +----------------------+
                | + connect()          |
                | + disconnect()       |
                | + moveJ()            |
                | + moveL()            |
                | + moveServo()        |
                | + moveLine()         |
                | + set_digital_output |
                | + get_state()        |
                | + emergency_stop()   |
                +----------+-----------+
                           | 實作 / 封裝
                           v
        +--------------------------------------+
        |  ROKAE_CR7_適配器 / JAKA_Zu7_適配器   |
        +--------------------------------------+
        |  + 品牌特定 SDK 轉換                   |
        +--------------------------------------+
```

---

3. 模組規格

3.1 核心模組

3.1.1 VirtualRobot 類別

目的: 用純 Python 模擬單一機械臂,API 貼近真實廠商 SDK。

建構函數:

```
VirtualRobot(
    robot_id: str,
    brand: str,         # "ROKAE", "JAKA", "UR", "ABB", ...
    model: str,         # "CR7", "Zu7", "UR5e", ...
    ip: str = "127.0.0.1",   # 虛擬 IP 占位
    spec: RobotSpec | None = None   # 可選覆寫規格
)
```

如果冇提供 `spec`,class 會由 `(brand, model)` 索引內建規格表。

實例變數:

變數	類型	說明
`robot_id`	str	唯一識別碼
`brand`	str	品牌名
`model`	str	型號
`ip`	str	網絡地址(虛擬占位)
`joints`	np.ndarray (6,)	當前關節角度(度)
`pose`	np.ndarray (6,)	當前笛卡爾位姿 [x, y, z, rx, ry, rz]
`io_states`	dict	數位 I/O 狀態 `{0: False, 1: False, ...}`
`connected`	bool	連線狀態
`e_stopped`	bool	緊急停止旗標
`spec`	RobotSpec	關節極限、負載、工作半徑

公開方法:

方法	簽名	說明	
`connect`	`(ip: str | None = None) -> bool`	建立連線	
`disconnect`	`() -> None`	中斷連線	
`moveJ`	`(joints: list[float]) -> None`	關節空間動作	
`moveL`	`(pose: list[float]) -> None`	笛卡爾直線動作	
`moveServo`	`(joints: list[float], t: float) -> None`	ROKAE 風格伺服動作	
`moveLine`	`(pose: list[float]) -> None`	ROKAE 風格直線動作	
`set_digital_output`	`(pin: int, value: bool) -> None`	設定 DOUT	
`get_digital_input`	`(pin: int) -> bool`	讀取 DIN	
`get_state`	`() -> RobotState`	取得當前狀態快照	
`emergency_stop`	`() -> None`	即時停止(設定 `e_stopped`)	

方法	簽名	說明
`reset_estop`	`() -> None`	清除緊急停止
`move_to_home`	`() -> None`	移動到原點 `[0, -90, 0, -90, 0, 0]`

自動執行嘅安全檢查:

1. 關節極限:每個關節角度必須喺 `±180°` 範圍內
2. 工作範圍:目標笛卡爾位置必須喺工作半徑內
3. 緊急停止檢查:如 `e_stopped` 為 True,拒絕動作
4. 連線檢查:未連線時拒絕動作

3.1.2 RobotManager 類別

目的: 同時管理 2-4 個機械人,支援協調動作同集中安全。

建構函數:

```
RobotManager()  # 空的管理器
```

公開方法:

方法	簽名	說明
`add_robot`	`(robot_id, brand, model, ip=None) -> VirtualRobot`	註冊機械人
`remove_robot`	`(robot_id) -> None`	移除註冊
`connect_all`	`() -> None`	連接所有機械人
`disconnect_all`	`() -> None`	中斷所有機械人
`get_robot`	`(robot_id) -> VirtualRobot`	取得單一機械人
`list_robots`	`() -> list[str]`	列出機械人 ID
`emergency_stop_all`	`() -> None`	即時停止所有機械人
`reset_estop_all`	`() -> None`	清除全部緊急停止
`sync_move`	`(commands: dict[str, dict]) -> None`	同步動作
`status`	`() -> dict`	取得全部機械人狀態摘要

sync_move 範例:

```
manager.sync_move({
    "arm_1": {"moveJ": [0, -45, 0, -90, 0, 0]},
    "arm_2": {"moveL": [0.4, 0.0, 0.5, 0, 0, 0]},
})
```

所有機械人同時開始動作。即時同步容忍度 10ms 之內,做法係每個機械人嘅指令喺獨立 thread 執行,主 thread join 全部。

3.1.3 RobotSpec 資料類別

```
@dataclass
class RobotSpec:
    brand: str
    model: str
    payload_kg: float
    reach_mm: float
    dof: int                      # 通常 6
    joint_limits_deg: list[tuple[float, float]]  # [(min, max), ...]
    max_joint_speed_dps: float  # 度/秒
    max_tcp_speed_mms: float    # TCP 直線速度 mm/s
    protection_rating: str      # "IP54", "IP67", ...
    weight_kg: float
```

內建規格: `ROKAE_CR7_SPEC` , `JAKA_ZU7_SPEC` 。

3.2 品牌適配器

3.2.1 ROKAE_CR7_適配器 (主要)

狀態: 完整規格,主要目標。

兩種實作:

類別	後端	用途
`VirtualRokaeRobot`	純 Python	寫字樓開發
`RealRokaeRobot`	ROKAE xCore SDK	工廠部署

兩個類別實作同一個公開 API。切換只需改一行 import。

ROKAE xCore API 對應:

RobotArm 方法	ROKAE xCore SDK 呼叫
`moveServo(joints, t)`	`robot.moveServo(joints, t)`

RobotArm 方法	ROKAE xCore SDK 呼叫
`moveLine(pose)`	`robot.moveLine(pose)`
`set_digital_output(pin, value)`	`robot.setDO(pin, value)`
`emergency_stop()`	`robot.emergencyStop()`
`get_state()`	`robot.getRobotState()`

ROKAE CR7 規格值(請對照 xCore 手冊驗證):

項目	值
負載	7 kg
工作半徑	988 mm
自由度	6
關節極限	每個 ±180°(需驗證)
重複精度	±0.02 mm
防護等級	IP67
重量	~ 27 kg

> **注意:** 確切嘅關節極限同最大速度必須對照 ROKAE xCore 用戶手冊驗證。本文件假設通用嘅 ±180° 極限。

3.2.2 JAKA_Zu7_適配器 (次要)

狀態: 概要,次要目標。

同樣嘅雙 class 模式:

類別	後端
`VirtualJakaRobot`	純 Python
`RealJakaRobot`	JAKA Python SDK

JAKA API 對應(規劃中):

RobotArm 方法	JAKA SDK 呼叫
`moveJ(joints)`	`robot.joint_move(joints)`
`moveL(pose)`	`robot.linear_move(pose)`
`set_digital_output`	`robot.set_digital_output`

RobotArm 方法	JAKA SDK 呼叫
`emergency_stop`	`robot.emergency_stop`

JAKA Zu7 規格值(請對照 JAKA 手冊驗證):

項目	值
負載	7 kg
工作半徑	819 mm
自由度	6
防護等級	IP54
重量	~ 12 kg

3.2.3 未來品牌

適配器模式支援透過以下步驟加入新品牌:

1. 為新型號加入 `BRAND_SPECS` 項目
2. 建立 `Adapter` class 實作同一個 API
3. (可選) 提供 `RealAdapter` 封裝廠商 SDK

未來候選:**Universal Robots (UR5e, UR10e)**、**ABB GoFa**、**Fanuc CRX**、**Doosan**。

3.3 應用模組

3.3.1 welding.py (主要應用)

狀態: 主要目標應用。

公開函數:

函數	說明
`run_linear_weld(robot, start_pose, end_pose, speed=0.1)`	兩點之間嘅直線焊接
`run_weave_weld(robot, path, amplitude, frequency)`	擺動焊接
`torch_on(robot, pin=0)`	啟動焊槍
`torch_off(robot, pin=0)`	關閉焊槍
`set_welding_params(robot, current, voltage)`	透過 DO/類比設定焊接參數

範例:

```
from robotarm import VirtualRobot
from robotarm.applications import welding

robot = VirtualRobot(robot_id="weld_1", brand="ROKAE", model="CR7")
robot.connect()
robot.move_to_home()

start = [0.4, -0.1, 0.3, 0, 0, 0]
end   = [0.4,  0.1, 0.3, 0, 0, 0]

torch_on(robot)
run_linear_weld(robot, start, end, speed=0.1)
torch_off(robot)

robot.disconnect()
```

3.3.2 pick_place.py (概要)

- `pick(robot, target_pose)` — 夾爪接近 + 夾緊
- `place(robot, target_pose)` — 夾爪接近 + 鬆開
- `pick_and_place(robot, from_pose, to_pose)` — 組合動作

3.3.3 assembly.py (概要)

- `precision_move(robot, pose, tolerance=0.001)` — 高精度動作
- `force_guided_move(...)` — 力控接口(將來 SDK 依賴)

3.3.4 inspection.py (概要)

- `follow_inspection_path(robot, waypoints, sensor_callback)` — 帶傳感器觸發嘅路徑

3.4 工具模組

3.4.1 kinematics.py

```
def forward_kinematics(
    robot: VirtualRobot,
    joints: list[float]
) -> np.ndarray:
    """由關節角度計算 TCP 位姿 (返回 [x,y,z,rx,ry,rz])"""

def inverse_kinematics(
    robot: VirtualRobot,
    pose: np.ndarray,
```

```
    seed_joints: list[float] | None = None
) -> np.ndarray:
    """由目標位姿反解關節角度. 可選 - 真實機械人可交畀廠商 SDK"""
```

虛擬實作使用 **Denavit-Hartenberg (DH) 參數**。真實機械人嘅 IK 可以交畀廠商 SDK。

3.4.2 safety.py

```
def check_joint_limits(robot: VirtualRobot, joints: list[float]) ->
tuple[bool, str]
def check_reach(robot: VirtualRobot, pose: list[float]) -> tuple[bool,
str]
def check_collision(robots: list[VirtualRobot]) -> tuple[bool, str]
def enforce_safety(robot: VirtualRobot, action: Callable) -> None
```

3.4.3 logger.py

集中日誌配置:

```
from robotarm.utils.logger import get_logger

logger = get_logger("robotarm.welding")
logger.info("由 %s 開始焊接去 %s", start, end)
```

支援可配置等級(DEBUG / INFO / WARNING / ERROR)、console + file 輸出,以及用於生產嘅結構化日誌。

---

4. API 參考

4.1 `VirtualRobot`

`connect(ip: str | None = None) -> bool`

建立(虛擬)連線。

- **參數:** `ip` — 可選 IP 覆寫;預設係建構函數 IP
- **返回:** 成功返回 `True`
- **異常:** 已連線時拋出 `RobotAlreadyConnectedError`

`moveJ(joints: list[float]) -> None`

關節空間動作(點對點)。

- **參數:** `joints` — 6 元素嘅關節角度列表(度)
- **異常:** `JointLimitError`、`NotConnectedError`、`EmergencyStopError`

`moveL(pose: list[float]) -> None`

笛卡爾直線動作。

- **參數:** `pose` — `[x, y, z, rx, ry, rz]`,單位米同弧度
- **異常:** `ReachError`、`NotConnectedError`、`EmergencyStopError`

`moveServo(joints: list[float], t: float) -> None`

ROKAE 風格嘅伺服(串流)動作。

- **參數:** `joints`、`t` — 持續時間(秒)

`moveLine(pose: list[float]) -> None`

ROKAE 風格直線動作到位姿。

`set_digital_output(pin: int, value: bool) -> None`

設定數位輸出腳位。

`get_state() -> RobotState`

返回 `RobotState` 資料類別,包含 `joints`、`pose`、`io_states`、`connected`、`e_stopped`。

`emergency_stop() -> None`

設定 `e_stopped=True`。所有後續動作會被拒絕,直至 `reset_estop()`。

4.2 `RobotManager`

`add_robot(robot_id, brand, model, ip=None) -> VirtualRobot`

註冊機械人,返回建立嘅 `VirtualRobot`。

`emergency_stop_all() -> None`

停止所有機械人,thread-safe。

`sync_move(commands: dict) -> None`

並行執行動作。

- **參數:** `commands` — `{robot_id: {"method_name": args}}`
- **行為:** 每個指令喺獨立 thread 執行;主 thread join 全部。

4.3 異常

異常	觸發時機
`NotConnectedError`	`connect()` 前嘗試動作
`RobotAlreadyConnectedError`	重複呼叫 `connect()`
`JointLimitError`	關節角度超出 ±180°
`ReachError`	目標超出工作半徑
`EmergencyStopError`	緊急停止狀態下嘗試動作
`InvalidPoseError`	位姿 shape 錯 / 非有限值

---

5. 使用案例

5.1 單一機械人虛擬測試

```
from robotarm import VirtualRobot

robot = VirtualRobot(robot_id="weld_1", brand="ROKAE", model="CR7")
robot.connect("192.168.1.100")          # 虛擬 IP 占位
robot.move_to_home()
robot.move_to_pose([0.4, 0.0, 0.5, 0, 0, 0])
robot.set_digital_output(0, True)
robot.disconnect()
```

5.2 多機械人協調動作

```
from robotarm import RobotManager

manager = RobotManager()
manager.add_robot("arm_1", brand="ROKAE", ip="192.168.1.100")
manager.add_robot("arm_2", brand="ROKAE", ip="192.168.1.101")
manager.connect_all()
manager.emergency_stop_all()
manager.disconnect_all()
```

5.3 焊接應用

```
from robotarm import VirtualRobot
from robotarm.applications import welding

robot = VirtualRobot(robot_id="weld_1", brand="ROKAE", model="CR7")
robot.connect("192.168.1.100")
welding.run_linear_weld(robot, start_pose, end_pose, speed=0.1)
robot.disconnect()
```

5.4 由虛擬切換去真實

```
# ----- 寫字樓開發 -----
from robotarm import VirtualRobot
robot = VirtualRobot(robot_id="weld_1", brand="ROKAE", model="CR7")
robot.connect("192.168.1.100")          # 虛擬

# ----- 工廠部署(只改 import)-----
from robotarm.brands.rokae import RealRokaeRobot
robot = RealRokaeRobot()
robot.connect("192.168.1.100")          # 真實 SDK
```

---

6. 專案結構

```
/home/ubuntu/RobotArm/
|-- README.md
|-- LICENSE
|-- pyproject.toml
|-- requirements.txt
|-- setup.cfg
|-- robotarm/
|   |-- __init__.py
|   |-- core/
|   |   |-- __init__.py
|   |   |-- virtual_robot.py
|   |   |-- robot_manager.py
|   |   `-- robot_spec.py
|   |-- brands/
|   |   |-- __init__.py
|   |   |-- registry.py             # BRAND_SPECS 查詢
|   |   |-- rokae_adapter.py        # VirtualRokaeRobot +
RealRokaeRobot
|   |   `-- jaka_adapter.py         # VirtualJakaRobot + RealJakaRobot
|   |-- applications/
```

```
|   |   |-- __init__.py
|   |   |-- welding.py
|   |   |-- pick_place.py
|   |   |-- assembly.py
|   |   `-- inspection.py
|   |-- utils/
|   |   |-- __init__.py
|   |   |-- kinematics.py
|   |   |-- safety.py
|   |   `-- logger.py
|   `-- exceptions.py
|-- examples/
|   |-- 01_single_robot.py
|   |-- 02_multi_robot.py
|   |-- 03_welding.py
|   `-- 04_virtual_to_real.py
|-- tests/
|   |-- test_virtual_robot.py
|   |-- test_robot_manager.py
|   |-- test_kinematics.py
|   `-- test_safety.py
|-- docs/
|   |-- api.md
|   |-- installation.md
|   |-- quickstart.md
|   `-- troubleshooting.md
`-- config/
    `-- default.yaml
```

命名慣例:

- 模組: `snake_case`
- 類別: `PascalCase`
- 方法 / 變數: `snake_case`
- 常數: `UPPER_SNAKE_CASE`

---

7. 開發路線圖

階段	週數	目標	交付物
1	1-4	虛擬核心	ROKAE CR7 嘅 `VirtualRobot`、正向運動學、安全檢查、demo apps

階段	週數	目標	交付物
2	5-6	多機械人支援	`RobotManager`、同步動作、用 2-4 個虛擬機械人測試
3	7-8	真實 SDK 整合	ROKAE xCore 整合、工廠實體 CR7 測試、虛擬對真實驗證
4	9-10	JAKA 支援	`JAKA_Zu7_Adapter`、實體 Zu7 測試、多品牌協調
5	11-14	應用模組	完成焊接、加上下料、加組裝、範例
6	15-16	文件同測試	API 文件、單元測試、教學、bug 修復、優化

預計總工期: 16 週(4 個月)。

---

8. 風險評估

8.1 技術風險

風險	影響	機會率	緩解措施
正向運動學準確度低過真實機械人	高	中	用廠商 DH 參數校準;對 SDK 做位姿對標
多機械人同步超出 10ms	中	中	用 threading + thread barriers;做 benchmark
虛擬 API 同廠商 SDK 偏離	高	低	預先用廠商文件定義 API;整合測試
即時性能瓶頸	中	低	對熱路徑做 profile;用 numpy 向量化
廠商 SDK 版本錯配	中	中	喺 `requirements.txt` pin SDK 版本;CI 整合測試

8.2 安全風險

風險	影響	機會率	緩解措施
緊急停止無法停止動作	嚴重	低	獨立緊急停止線路(硬件);軟件緊急停止作為後備
兩個機械人互相碰撞	高	低	維護碰撞區地圖;預先檢查動作路徑
違反關節極限損壞硬件	高	低	每個動作指令預先驗證;軟 + 硬極限

風險	影響	機會率	緩解措施
動作中途斷線	高	低	心跳監測;斷線時自動緊急停止
焊槍卡住開住	中	低	焊槍 watchdog 連超時;冗餘關閉訊號

8.3 緩解策略摘要

1. **雙層安全:** 軟件安全檢查(強制) + 硬件緊急停止(永遠存在)
2. **漸進式推出:** 虛擬 → 單一真實 → 雙真實 → 多品牌
3. **持續驗證:** 並行執行虛擬同真實,記錄位姿差異
4. **文件紀律:** 每個 API 改動先文件化先 merge

---

9. 成功準則

準則	量度
虛擬機械人準確模擬真實機械人	相同指令下,位姿差異 < 1mm(對比廠商 SDK)
寫字樓寫嘅 code 第一次喺工廠用就 work	>= 90% 寫字樓寫嘅程式喺真實機械人上唔改直接 run
多機械人協調流暢	2-4 個機械人同步容忍度 < 10ms
容易加入新機械人品牌	加入新品牌 <= 1 日(規格 + 適配器 + 測試)
文件同範例清楚	公開方法全部有文件;>= 4 個可運行範例
零意外安全事故	硬件 + 軟件緊急停止每週驗證

附錄 A — 機械人規格參考(預填)

> **使用前請對照廠商手冊驗證。**

ROKAE CR7

項目	值
負載	7 kg
工作半徑	988 mm
自由度	6
重複精度	±0.02 mm

項目	值
防護等級	IP67
重量	~ 27 kg
關節範圍	~ ±360°(每個關節需驗證)

JAKA Zu7

項目	值
負載	7 kg
工作半徑	819 mm
自由度	6
防護等級	IP54
重量	~ 12 kg
關節範圍	~ ±360°(每個關節需驗證)

附錄 B — 詞彙表

術語	定義
TCP	Tool Center Point(工具中心點)— 機械人末端嘅動作命令點
Pose	笛卡爾位置 + 方向 `[x, y, z, rx, ry, rz]`
DH 參數	Denavit-Hartenberg 參數,用於正向運動學
Cobot	Collaborative robot(協作機械人)— 設計畀人機安全互動
E-stop	Emergency stop(緊急停止)
SDK	Software Development Kit(軟件開發套件)
關節空間	用關節角度定義嘅動作
笛卡爾空間	用 TCP 位姿定義嘅動作

附錄 C — 文件生成備註

> 本 SDD 由 Hermes Agent 根據用戶嘅規格 prompt 生成。3.2 同 4 節嘅 ROKAE xCore 同 JAKA Python SDK 方法名係基於常見嘅公開命名慣例,**實作前必須對照官方廠商**

> **SDK 文件驗證**。附錄 A 嘅硬件規格亦應對照廠商 datasheet 確認。

文件完